

Update and Delete with Flask-RESTful (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-flask-restful-ud>  <https://github.com/learn-co-curriculum/python-p4-flask-restful-ud/issues/new>

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

Now that we've explored how to create and retrieve records with `POST` and `GET`, let's look into updating records individually and in batches with `PATCH`.

Run `pipenv install && pipenv shell` to enter your virtual environment.

```
$ pipenv install && pipenv shell
```

Change into the `server` directory to create the database and populate it with seed data:

```
$ cd server
$ flask db upgrade
$ python seed.py
```

Adding a `PATCH` Route

We're just about pros with Flask-RESTful now: if you're feeling confident, go ahead and try adding a `patch()` method to `NewsletterByID`.

****TIP:** Remember to use `setattr()` to cut down the number of lines of code!

Solution below...

...

...

...

Just as with the `get()` route in this resource, we're going to pass in the `id` from the URL and use it to retrieve the record we're updating. Unlike the `get()` route, we want to leave it as a `Newsletter` object instead of converting it to a dictionary, since we want to change the attributes on the record.

```
# server/app.py

class NewsletterByID(Resource):

    # get()

    def patch(self, id):

        record = Newsletter.query.filter(Newsletter.id == id).first()
        for attr in request.form:
            setattr(record, attr, request.form[attr])

        db.session.add(record)
        db.session.commit()

        response_dict = record.to_dict()

        response = make_response(
            response_dict,
            200
        )

        return response
```

Looping through the form data gives us its keys, the attribute names to be changed. From there, we can set each attribute on the `Newsletter` object to its new value with `setattr()`. From here, we update the database with the new record, create a response with the record, and send it back to the client.

Try it out for yourself: open Postman and navigate to <http://127.0.0.1:5000/newsletters/20>  (<http://127.0.0.1:5000/newsletters/20>). Change the request method to `PATCH`, edit the `Body > form-data` with a new body, then hit submit. You should see a response similar to the following:

```
{  
  "body": "blah blah blah blah blah blah blah",  
  "edited_at": "2022-09-22 16:50:06",  
  "id": 20,  
  "published_at": "2022-09-22 16:48:12",  
  "title": "Mr. Title"  
}
```

Looks like someone didn't enjoy the newsletter from September 22!

Adding a **DELETE** Route

Now that we've identified an unpopular newsletter, it might be a good idea to head into the API and delete it. To accomplish that, we'll have to add a `delete()` route to `NewsletterByID`:

```
# server/app.py
```

```
class NewsletterByID(Resource):  
  
    # get(), patch()  
  
    def delete(self, id):  
  
        record = Newsletter.query.filter(Newsletter.id == id).first()  
  
        db.session.delete(record)  
        db.session.commit()  
  
        response_dict = {"message": "record successfully deleted"}  
  
        response = make_response(  

```

```
        response_dict,  
        200  
    )  
  
    return response
```

Here, we retrieve the record using the `id` passed to the route through the URL, delete it, and return a message that the record has been successfully deleted:

```
{  
    "message": "record successfully deleted"  
}
```

Conclusion

While there are a few nuances that separate Flask-RESTful from vanilla Flask in building APIs, we can see that they exist to simplify the process of making a RESTful API: rather than specifying the accepted HTTP request methods and handling them with `if/elif/else` blocks, methods are used to neatly organize the routes for `GET`, `POST`, `PATCH`, `DELETE`, and more at each URL.

Solution Code

```
# server/app.py  
  
#!/usr/bin/env python3  
  
from flask import Flask, request, make_response  
from flask_migrate import Migrate  
from flask_restful import Api, Resource
```

```
from models import db, Newsletter

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///newsletters.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)
db.init_app(app)

api = Api(app)

class Index(Resource):

    def get(self):

        response_dict = {
            "index": "Welcome to the Newsletter RESTful API",
        }

        response = make_response(
            response_dict,
            200,
        )

        return response

api.add_resource(Index, '/')

class Newsletters(Resource):

    def get(self):
```

```
response_dict_list = [n.to_dict() for n in Newsletter.query.all()]

response = make_response(
    response_dict_list,
    200,
)

return response

def post(self):

    new_record = Newsletter(
        title=request.form['title'],
        body=request.form['body'],
    )

    db.session.add(new_record)
    db.session.commit()

    response_dict = new_record.to_dict()

    response = make_response(
        response_dict,
        201,
    )

    return response

api.add_resource(Newsletters, '/newsletters')

class NewsletterByID(Resource):

    def get(self, id):
```

```
response_dict = Newsletter.query.filter_by(id=id).first().to_dict()

response = make_response(
    response_dict,
    200,
)

return response

def patch(self, id):

    record = Newsletter.query.filter_by(id=id).first()
    for attr in request.form:
        setattr(record, attr, request.form[attr])

    db.session.add(record)
    db.session.commit()

    response_dict = record.to_dict()

    response = make_response(
        response_dict,
        200
    )

    return response

def delete(self, id):

    record = Newsletter.query.filter_by(id=id).first()

    db.session.delete(record)
```



```
db.session.commit()

response_dict = {"message": "record successfully deleted"}

response = make_response(
    response_dict,
    200
)

return response

api.add_resource(NewsletterByID, '/newsletters/<int:id>')

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means)  (<https://codewords.recurse.com/issues/five/what-restful-actually-means>)
- [Flask-RESTful](https://flask-restful.readthedocs.io/en/latest/)  (<https://flask-restful.readthedocs.io/en/latest/>)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)  (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>)

This tool needs to be loaded in a new browser window

Load Update and Delete with Flask-RESTful (CodeGrade) in a new window